Python 笔试高频 · TOP5 类型 × 每类 TOP5 题

> 面向**笔试 / 机试**整理：精选 5 个最常考的类型，每类 5 道高频题。 每题包含「题意」「考点」「思路」「常规写法」「最 Pythonic 写法」「复杂度」。 命名遵循 Python 风格：函数用 `snake_case`，参数取语义化名字（`nums` / `words` / `coins` 等）。 题面均为自写简述，代码可直接运行。

目录

#	类型	高频题
一	数组（Array）	最大子数组和 / 合并区间 / 移动零 / 旋转数组 / 除自身外乘积
二	哈希表（Hash Table）	两数之和 / 字母异位词分组 / 最长连续序列 / 只出现一次的数字 / 有效字母异位词
三	字符串 / 序列（String）	最长无重复子串 / 最长公共前缀 / 反转单词 / 验证回文 / 字符串相加
四	栈与队列（Stack & Queue）	有效括号 / 每日温度 / 最小栈 / 逆波兰求值 / 栈实现队列
五	动态规划（Dynamic Programming）	爬楼梯 / 打家劫舍 / 零钱兑换 / 最长递增子序列 / 不同路径

速记口诀

- **数组**：双指针原地改、前缀和换时间、排序后再贪心。
- **哈希**：「存在 / 计数 / 配对」一律 O(1) 查，`dict` / `set` / `Counter` 三件套。
- **字符串**：能 `split` / 切片 / `Counter` 就别手撸；滑动窗口治连续子串。
- **栈队列**：括号匹配、下一个更大元素（单调栈）、双栈互转。
- **动态规划**：定义状态 → 写转移 → 定初值与顺序 → 滚动变量压空间。

类型一：数组（Array）

1. 最大子数组和（Maximum Subarray，LC 53，Medium）

题意：求和最大的**连续**子数组，返回其和。

考点：Kadane 动态规划——「以当前元素结尾的最大和」。

思路：维护 `current`（以当前元素结尾的最大和），`current = max(num, current + num)`；全局答案取所有 `current` 的最大值。

常规写法：

```
def max_sub_array(nums: list[int]) -> int:
    best = current = nums[0]
    for num in nums[1:]:
```

```
        current = max(num, current + num)
        best = max(best, current)
    return best
```

最 Pythonic 写法：把递推交给 `accumulate` 的自定义二元函数，一行求最大。

```
from itertools import accumulate

def max_sub_array(nums: list[int]) -> int:
    return max(accumulate(nums, lambda current, num: max(num, current + num)))
```

复杂度：时间 O(n)，空间 O(1)。

---

2. 合并区间（Merge Intervals，LC 56，Medium）

题意：合并所有有重叠的区间。

考点：按起点排序后贪心扫描。

思路：先按起点排序；遍历时若当前区间起点 ≤ 结果末尾区间的终点则重叠，更新终点取较大者，否则新开一个区间。

常规写法：用下标遍历，单独处理空输入与首区间。

```
def merge(intervals: list[list[int]]) -> list[list[int]]:
    if not intervals:
        return []
    intervals.sort(key=lambda interval: interval[0])
    merged = [intervals[0]]
    for i in range(1, len(intervals)):
        if intervals[i][0] <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], intervals[i][1])
        else:
            merged.append(intervals[i])
    return merged
```

最 Pythonic 写法：`sorted` 默认按首元素排，元组解包遍历，`merged and` 自然处理空输入。

```
def merge(intervals: list[list[int]]) -> list[list[int]]:
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
```

```
            merged.append([start, end])
    return merged
```

复杂度：时间 O(n log n)，空间 O(n)。

---

3. 移动零（Move Zeroes，LC 283，Easy）

题意：把所有 `0` 移到末尾，保持非零元素相对顺序，**原地**修改。

考点：双指针原地覆盖。

思路：慢指针 `slow` 指向下一个待放置位置，快指针遇到非零就和 `slow` 交换并右移 `slow`。

常规写法：双指针交换，O(1) 额外空间。

```
def move_zeroes(nums: list[int]) -> None:
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

最 Pythonic 写法：列表推导挑出非零，再用切片赋值补零（额外 O(n) 空间换可读性）。

```
def move_zeroes(nums: list[int]) -> None:
    non_zeros = [num for num in nums if num != 0]
    nums[:] = non_zeros + [0] * (len(nums) - len(non_zeros))
```

复杂度：双指针 O(n) / O(1)；推导版 O(n) / O(n)。

---

4. 旋转数组（Rotate Array，LC 189，Medium）

题意：将数组向右轮转 `k` 个位置，**原地**修改。

考点：三次反转（O(1) 空间）/ 切片重组。

思路：先整体反转，再分别反转前 `k` 个与剩余部分；或直接用切片把末尾 `k` 个搬到前面。

常规写法：三次反转，O(1) 额外空间。

```
def rotate(nums: list[int], k: int) -> None:
    n = len(nums)
    k %= n

    def reverse(lo: int, hi: int) -> None:
        while lo < hi:
```

```
            nums[lo], nums[hi] = nums[hi], nums[lo]
            lo += 1
            hi -= 1

    reverse(0, n - 1)
    reverse(0, k - 1)
    reverse(k, n - 1)
```

最 Pythonic 写法：切片拼接一行完成（`k = 0` 时 `nums[-0:]` 为整段、`nums[:-0]` 为空，结果仍正确）。

```
def rotate(nums: list[int], k: int) -> None:
    k %= len(nums)
    nums[:] = nums[-k:] + nums[:-k]
```

复杂度：反转法 O(n) / O(1)；切片法 O(n) / O(n)。

---

5. 除自身以外数组的乘积（Product of Array Except Self，LC 238，Medium）

题意：返回数组 `answer`，其中 `answer[i]` 等于除 `nums[i]` 外其余元素的乘积，**不能用除法**。

考点：前缀积 × 后缀积。

思路：先把每个位置左侧乘积存入 `answer`，再从右往左乘上右侧乘积。

常规写法：两趟扫描，前缀积与后缀积各一遍。

```
def product_except_self(nums: list[int]) -> list[int]:
    n = len(nums)
    answer = [1] * n
    prefix = 1
    for i in range(n):
        answer[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix
        suffix *= nums[i]
    return answer
```

最 Pythonic 写法：用 `accumulate(..., mul)` 分别求前缀积与后缀积，再逐位相乘。

```
from itertools import accumulate
from operator import mul

def product_except_self(nums: list[int]) -> list[int]:
```

```
    prefix = accumulate([1] + nums[:-1], mul)            # 每位左侧乘积
    suffix = list(accumulate([1] + nums[:0:-1], mul))[::-1]   # 每位右侧乘积
    return [p * s for p, s in zip(prefix, suffix)]
```

复杂度：时间 O(n)，空间 O(1)（不计输出）。

类型二：哈希表（Hash Table）

1. 两数之和（Two Sum，LC 1，Easy）

题意：返回数组中和为 `target` 的两个下标。

考点：哈希表记「值 → 下标」，边遍历边查补数。

思路：对当前 `num`，查 `target - num` 是否已出现，命中即返回两个下标。

常规写法：

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {}                           # 值 -> 下标
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

最 Pythonic 写法：用 `dict.get` 配合海象运算符，一次查询替代「先判断再取值」。

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {}
    for i, num in enumerate(nums):
        if (j := seen.get(target - num)) is not None:
            return [j, i]
        seen[num] = i
    return []
```

复杂度：时间 O(n)，空间 O(n)。

---

2. 字母异位词分组（Group Anagrams，LC 49，Medium）

题意：把由相同字母重排得到的字符串归到一组。

考点：为异位词设计唯一 key（排序串 / 字符计数）。

思路：异位词排序后相同，用 `''.join(sorted(word))` 作 key 聚合。

常规写法：普通字典 + 手动判断 key 是否存在。

```
def group_anagrams(words: list[str]) -> list[list[str]]:
    groups = {}
    for word in words:
        key = ''.join(sorted(word))
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())
```

最 Pythonic 写法：`defaultdict(list)` 免去存在性判断。

```
from collections import defaultdict

def group_anagrams(words: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for word in words:
        groups[''.join(sorted(word))].append(word)
    return list(groups.values())
```

复杂度：时间 O(n·L log L)，空间 O(n·L)。

---

3. 最长连续序列（Longest Consecutive Sequence，LC 128，Medium）

题意：找最长**连续整数**序列的长度，要求 O(n)。

考点：哈希集合 + 只从「序列起点」向上扩展。

思路：把所有数放进 `set`；只有当 `num - 1` 不在集合中（说明 `num` 是某段起点）时才向上数长度，保证每个数最多被访问两次。

常规写法：

```
def longest_consecutive(nums: list[int]) -> int:
    num_set = set(nums)
    longest = 0
    for num in num_set:
        if num - 1 not in num_set:          # num 是某段起点
            length = 1
            while num + length in num_set:
                length += 1
            longest = max(longest, length)
    return longest
```

最 Pythonic 写法：把「数长度」抽成内层函数，用生成器表达式求最大值。

```python
def longest_consecutive(nums: list[int]) -> int:
    num_set = set(nums)

    def streak_from(start: int) -> int:
        length = 1
        while start + length in num_set:
            length += 1
        return length

    starts = (num for num in num_set if num - 1 not in num_set)
    return max((streak_from(num) for num in starts), default=0)
```

复杂度：时间 O(n)，空间 O(n)。

---

4. 只出现一次的数字（Single Number，LC 136，Easy）

题意：数组中每个元素出现两次、仅一个出现一次，找出它。要求线性时间。

考点：异或位运算（成对抵消）/ 哈希计数。

思路：`a ^ a = 0`、`a ^ 0 = a`，把所有数异或起来，成对的相互抵消，剩下答案。

常规写法：哈希计数，通用直观。

```python
def single_number(nums: list[int]) -> int:
    counts = {}
    for num in nums:
        counts[num] = counts.get(num, 0) + 1
    for num, freq in counts.items():
        if freq == 1:
            return num
    return -1
```

最 Pythonic 写法：`reduce` + `operator.xor` 归约，O(1) 空间。

```python
from functools import reduce
from operator import xor

def single_number(nums: list[int]) -> int:
    return reduce(xor, nums)
```

复杂度：异或法 时间 O(n)，空间 O(1)。

---

5. 有效的字母异位词（Valid Anagram，LC 242，Easy）

题意：判断 `target` 是否为 `source` 的字母异位词。

考点：字符频次比较。

思路：两串字符出现次数完全相同即互为异位词。

常规写法：手动维护计数字典，逐字符增减。

```python
def is_anagram(source: str, target: str) -> bool:
    if len(source) != len(target):
        return False
    counts = {}
    for ch in source:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in target:
        if ch not in counts:
            return False
        counts[ch] -= 1
        if counts[ch] == 0:
            del counts[ch]
    return not counts
```

最 Pythonic 写法：直接比较两个 `Counter`。

```python
from collections import Counter

def is_anagram(source: str, target: str) -> bool:
    return Counter(source) == Counter(target)
```

复杂度：时间 O(n)，空间 O(字符集)。

类型三：字符串 / 序列（String）

1. 最长无重复字符子串（Longest Substring Without Repeating，LC 3，Medium）

题意：求不含重复字符的最长子串长度。

考点：滑动窗口 + 哈希记录字符最近位置。

思路：右指针扩张，遇到重复字符时把左指针跳到该字符上次出现位置的下一格。

常规写法：哈希记录每个字符最近下标。

```python
def length_of_longest_substring(text: str) -> int:
    last_seen = {}                      # 字符 -> 最近下标
    left = longest = 0
    for right, ch in enumerate(text):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        longest = max(longest, right - left + 1)
    return longest
```

最 Pythonic 写法：用 `set` 维护当前窗口，遇重复从左收缩，语义更直白。

```python
def length_of_longest_substring(text: str) -> int:
    window = set()
    left = longest = 0
    for right, ch in enumerate(text):
        while ch in window:
            window.remove(text[left])
            left += 1
        window.add(ch)
        longest = max(longest, right - left + 1)
    return longest
```

复杂度：时间 O(n)，空间 O(字符集)。

---

2. 最长公共前缀（Longest Common Prefix，LC 14，Easy）

题意：查找字符串数组的最长公共前缀，不存在返回空串。

考点：纵向逐列比较 / `zip` 转置。

思路：逐列取所有字符串同一位置的字符，全部相同则计入前缀，出现不同即停止。

常规写法：以首串为基准，不断裁剪直到成为公共前缀。

```python
def longest_common_prefix(words: list[str]) -> str:
    if not words:
        return ""
    prefix = words[0]
    for word in words[1:]:
        while not word.startswith(prefix):
            prefix = prefix[:-1]
            if not prefix:
                return ""
    return prefix
```

最 Pythonic 写法：`zip(*words)` 逐列转置，`set` 判断该列是否一致。

```python
def longest_common_prefix(words: list[str]) -> str:
    prefix = []
    for column in zip(*words):
        if len(set(column)) == 1:
            prefix.append(column[0])
        else:
            break
    return ''.join(prefix)
```

复杂度：时间 O(n·L)，空间 O(L)。

---

3. 反转字符串中的单词（Reverse Words in a String，LC 151，Medium）

题意：反转句子中单词的顺序，并去除多余空格。

考点：`split` 去空白 / 双指针手动解析。

思路：按空白切分出单词列表，反转后用单个空格拼接。

常规写法：手动扫描，遇空格切词。

```python
def reverse_words(sentence: str) -> str:
    words = []
    word = []
    for ch in sentence:
        if ch != ' ':
            word.append(ch)
        elif word:
            words.append(''.join(word))
            word = []
    if word:
        words.append(''.join(word))
    return ' '.join(reversed(words))
```

最 Pythonic 写法：`split()` 自动忽略连续空白，切片反转一行搞定。

```python
def reverse_words(sentence: str) -> str:
    return ' '.join(sentence.split()[::-1])
```

复杂度：时间 O(n)，空间 O(n)。

---

4. 验证回文串（Valid Palindrome，LC 125，Easy）

题意：只考虑字母和数字、忽略大小写，判断是否为回文。

考点：双指针对撞 / 过滤后切片比较。

思路：跳过非字母数字字符，对撞双指针逐对比较；或先过滤清洗再与反转比较。

常规写法：双指针，O(1) 额外空间。

```
def is_palindrome(text: str) -> bool:
    left, right = 0, len(text) - 1
    while left < right:
        while left < right and not text[left].isalnum():
            left += 1
        while left < right and not text[right].isalnum():
            right -= 1
        if text[left].lower() != text[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

最 Pythonic 写法：列表推导过滤清洗，直接与反转切片比较。

```
def is_palindrome(text: str) -> bool:
    cleaned = [ch.lower() for ch in text if ch.isalnum()]
    return cleaned == cleaned[::-1]
```

复杂度：双指针 O(n) / O(1)；过滤版 O(n) / O(n)。

---

5. 字符串相加（Add Strings，LC 415，Easy）

题意：以字符串形式返回两个非负整数之和，**不能直接转 int**（大数模拟）。

考点：从末位逐位相加处理进位。

思路：两数末位对齐，逐位相加用 `divmod` 拆出进位与当前位。

常规写法：双下标从尾部向前，手动补零。

```
def add_strings(num1: str, num2: str) -> str:
    i, j = len(num1) - 1, len(num2) - 1
    carry = 0
    result = []
    while i >= 0 or j >= 0 or carry:
        digit1 = int(num1[i]) if i >= 0 else 0
        digit2 = int(num2[j]) if j >= 0 else 0
        carry, digit = divmod(digit1 + digit2 + carry, 10)
        result.append(str(digit))
```

```
            i -= 1
            j -= 1
        return ''.join(reversed(result))
```

最 Pythonic 写法：`zip_longest` 反向对齐补零，逐位 `divmod`。

```
from itertools import zip_longest

def add_strings(num1: str, num2: str) -> str:
    carry = 0
    result = []
    for d1, d2 in zip_longest(reversed(num1), reversed(num2), fillvalue='0'):
        carry, digit = divmod(int(d1) + int(d2) + carry, 10)
        result.append(str(digit))
    if carry:
        result.append('1')
    return ''.join(reversed(result))
```

复杂度：时间 O(max(m, n))，空间 O(max(m, n))。

类型四：栈与队列（Stack & Queue）

1. 有效的括号（Valid Parentheses，LC 20，Easy）

题意：判断只含 `()[]{}` 的字符串是否括号匹配有效。

考点：栈 + 哈希映射配对。

思路：左括号入栈，遇右括号检查栈顶是否为对应左括号。

常规写法：栈 + 配对字典。

```
def is_valid(brackets: str) -> bool:
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in brackets:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

最 Pythonic 写法：反复消去成对括号直到不能再消（极短，代价是最坏 $O(n^2)$）。

```python
def is_valid(brackets: str) -> bool:
    while '()' in brackets or '[]' in brackets or '{}' in brackets:
        brackets = brackets.replace('()', '').replace('[]', '').replace('{}', '')
    return not brackets
```

复杂度：栈法 O(n) / O(n)；消去法最坏 $O(n^2)$。

---

2. 每日温度（Daily Temperatures，LC 739，Medium）

题意：对每天求还要等几天才会遇到更高温度，没有则为 0。

考点：单调递减栈。

思路：栈里存「尚未找到更高温」的下标，温度递减；遇到更高温就不断出栈并结算天数差。

常规写法：栈存下标。

```python
def daily_temperatures(temperatures: list[int]) -> list[int]:
    answer = [0] * len(temperatures)
    stack = []                          # 存下标，对应温度递减
    for i, temp in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temp:
            prev = stack.pop()
            answer[prev] = i - prev
        stack.append(i)
    return answer
```

最 Pythonic 写法：栈存 `(下标, 温度)` 元组，免去反复二次索引。

```python
def daily_temperatures(temperatures: list[int]) -> list[int]:
    answer = [0] * len(temperatures)
    stack = []                          # 存 (下标, 温度)
    for i, temp in enumerate(temperatures):
        while stack and stack[-1][1] < temp:
            prev, _ = stack.pop()
            answer[prev] = i - prev
        stack.append((i, temp))
    return answer
```

复杂度：时间 O(n)，空间 O(n)。

---

3. 最小栈（Min Stack，LC 155，Medium）

题意：设计支持 `push` / `pop` / `top` 与 **O(1)** 取最小值 `getMin` 的栈。

考点：辅助栈 / 存（值，当前最小）元组。

思路：每个元素入栈时同时记录「到此为止的最小值」，取最小直接读栈顶。

常规写法：主栈 + 最小值辅助栈。

```
class MinStack:
    def __init__(self) -> None:
        self._stack = []
        self._mins = []

    def push(self, value: int) -> None:
        self._stack.append(value)
        self._mins.append(min(value, self._mins[-1]) if self._mins else value)

    def pop(self) -> None:
        self._stack.pop()
        self._mins.pop()

    def top(self) -> int:
        return self._stack[-1]

    def getMin(self) -> int:
        return self._mins[-1]
```

最 Pythonic 写法：单栈存（值，到此为止的最小值）元组。

```
class MinStack:
    def __init__(self) -> None:
        self._stack = []                # 存（值，到此为止的最小值）

    def push(self, value: int) -> None:
        current_min = min(value, self._stack[-1][1]) if self._stack else value
        self._stack.append((value, current_min))

    def pop(self) -> None:
        self._stack.pop()

    def top(self) -> int:
        return self._stack[-1][0]

    def getMin(self) -> int:
        return self._stack[-1][1]
```

复杂度：所有操作 O(1)。

---

4. 逆波兰表达式求值（Evaluate Reverse Polish Notation，LC 150，Medium）

题意：计算逆波兰（后缀）表达式的值，除法向零取整。

考点：栈 + 运算符映射。

思路：数字入栈，遇运算符弹出两数计算后把结果压回。

常规写法：`if-elif` 分支处理四则运算。

```
def eval_rpn(tokens: list[str]) -> int:
    operators = {'+', '-', '*', '/'}
    stack = []
    for token in tokens:
        if token in operators:
            right = stack.pop()
            left = stack.pop()
            if token == '+':
                stack.append(left + right)
            elif token == '-':
                stack.append(left - right)
            elif token == '*':
                stack.append(left * right)
            else:
                stack.append(int(left / right))     # 向零取整
        else:
            stack.append(int(token))
    return stack[0]
```

最 Pythonic 写法：用 `operator` 模块把运算符直接映射成函数。

```
import operator

def eval_rpn(tokens: list[str]) -> int:
    operations = {
        '+': operator.add,
        '-': operator.sub,
        '*': operator.mul,
        '/': lambda left, right: int(left / right),   # 向零取整
    }
    stack = []
    for token in tokens:
        if token in operations:
            right = stack.pop()
            left = stack.pop()
            stack.append(operations[token](left, right))
        else:
            stack.append(int(token))
    return stack[0]
```

复杂度：时间 O(n)，空间 O(n)。

5. 用栈实现队列（Implement Queue using Stacks，LC 232，Easy）

题意：仅用两个栈实现先进先出队列（`push` / `pop` / `peek` / `empty`）。

考点：输入栈 + 输出栈，倒栈一次，摊还 O(1)。

思路：`push` 进输入栈；`pop` / `peek` 时若输出栈为空，则把输入栈全部倒入输出栈，再从其顶部取。

常规写法：逐个 `pop` / `append` 倒栈。

```
class MyQueue:
    def __init__(self) -> None:
        self._in_stack = []
        self._out_stack = []

    def push(self, value: int) -> None:
        self._in_stack.append(value)

    def _transfer(self) -> None:
        if not self._out_stack:
            while self._in_stack:
                self._out_stack.append(self._in_stack.pop())

    def pop(self) -> int:
        self._transfer()
        return self._out_stack.pop()

    def peek(self) -> int:
        self._transfer()
        return self._out_stack[-1]

    def empty(self) -> bool:
        return not self._in_stack and not self._out_stack
```

最 Pythonic 写法：用 `extend(reversed(...))` 一次性倒栈，`clear` 清空。

```
class MyQueue:
    def __init__(self) -> None:
        self._in_stack = []
        self._out_stack = []

    def push(self, value: int) -> None:
        self._in_stack.append(value)

    def _transfer(self) -> None:
        if not self._out_stack:
            self._out_stack.extend(reversed(self._in_stack))
            self._in_stack.clear()

    def pop(self) -> int:
        self._transfer()
```

```
        return self._out_stack.pop()

    def peek(self) -> int:
        self._transfer()
        return self._out_stack[-1]

    def empty(self) -> bool:
        return not (self._in_stack or self._out_stack)
```

复杂度：摊还 O(1)，空间 O(n)。

类型五：动态规划（Dynamic Programming）

1. 爬楼梯（Climbing Stairs，LC 70，Easy）

题意：每次爬 1 或 2 阶，求爬到第 `n` 阶的不同方法数。

考点：斐波那契递推 `f(n) = f(n-1) + f(n-2)`。

思路：方法数等于斐波那契数列，用两个滚动变量即可，无需数组。

常规写法：显式处理基例 + 滚动。

```
def climb_stairs(n: int) -> int:
    if n <= 2:
        return n
    first, second = 1, 2
    for _ in range(3, n + 1):
        first, second = second, first + second
    return second
```

最 Pythonic 写法：元组同时赋值滚动，循环 `n` 次。

```
def climb_stairs(n: int) -> int:
    first = second = 1
    for _ in range(n):
        first, second = second, first + second
    return first
```

复杂度：时间 O(n)，空间 O(1)。

---

2. 打家劫舍（House Robber，LC 198，Medium）

题意：不能偷相邻房屋，求能偷到的最大金额。

考点：线性 DP `rob(i) = max(rob(i-1), rob(i-2) + nums[i])`。

思路：在每间房做「偷或不偷」的最优选择。

常规写法：开 `dp` 数组，O(n) 空间。

```python
def rob(nums: list[int]) -> int:
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]
    dp = [0] * len(nums)
    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])
    for i in range(2, len(nums)):
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])
    return dp[-1]
```

最 Pythonic 写法：两个滚动变量，元组赋值，O(1) 空间且免去基例。

```python
def rob(nums: list[int]) -> int:
    prev, current = 0, 0                # prev: 偷到 i-2 的最优; current: 偷到 i-1
的最优
    for money in nums:
        prev, current = current, max(current, prev + money)
    return current
```

复杂度：时间 O(n)，空间 O(1)。

---

3. 零钱兑换（Coin Change，LC 322，Medium）

题意：用最少的硬币数凑出金额 `amount`，凑不出返回 -1（硬币无限）。

考点：完全背包 DP `dp[x] = min(dp[x - coin] + 1)`。

思路：`dp[x]` 表示凑出金额 `x` 的最少硬币数，初始化为无穷大，`dp[0] = 0`。

常规写法：外层枚举硬币，内层正序刷金额。

```python
def coin_change(coins: list[int], amount: int) -> int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for x in range(coin, amount + 1):
            dp[x] = min(dp[x], dp[x - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

最 Pythonic 写法：内层用 `min` + 生成器表达式，`default` 处理无解。

```python
def coin_change(coins: list[int], amount: int) -> int:
    dp = [0] + [float('inf')] * amount
    for x in range(1, amount + 1):
        dp[x] = min((dp[x - coin] + 1 for coin in coins if coin <= x),
default=float('inf'))
    return dp[amount] if dp[amount] != float('inf') else -1
```

复杂度：时间 O(amount × len(coins))，空间 O(amount)。

---

4. 最长递增子序列（Longest Increasing Subsequence，LC 300，Medium）

题意：求最长**严格递增**子序列的长度。

考点：DP $O(n^2)$ / 贪心 + 二分 O(n log n)。

思路：维护 `tails`，`tails[i]` 表示长度为 `i+1` 的递增子序列的最小末尾；新元素用二分找到插入位，能接长就追加，否则替换。

常规写法：经典 $O(n^2)$ DP。

```python
def length_of_lis(nums: list[int]) -> int:
    if not nums:
        return 0
    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

最 Pythonic 写法：贪心 + `bisect_left`，O(n log n)。

```python
from bisect import bisect_left

def length_of_lis(nums: list[int]) -> int:
    tails = []
    for num in nums:
        pos = bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

复杂度：DP $O(n^2)$；贪心 + 二分 O(n log n)，空间 O(n)。

---

5. 不同路径（Unique Paths，LC 62，Medium）

题意：`rows × cols` 网格从左上走到右下，每次只能向右或向下，求路径总数。

考点：二维 DP / 组合数学。

思路：`dp[i][j] = dp[i-1][j] + dp[i][j-1]`；本质是从 `rows + cols - 2` 步里选 `rows - 1` 步向下，即组合数。

常规写法：二维 DP 累加。

```python
def unique_paths(rows: int, cols: int) -> int:
    dp = [[1] * cols for _ in range(rows)]
    for i in range(1, rows):
        for j in range(1, cols):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[rows - 1][cols - 1]
```

最 Pythonic 写法：直接用标准库组合数 `math.comb`。

```python
from math import comb

def unique_paths(rows: int, cols: int) -> int:
    return comb(rows + cols - 2, rows - 1)
```

复杂度：DP O(rows × cols)；组合数 O(min(rows, cols))。
